BACKEND

Diego Gomes de Santana

0 Roteiro da aula

1. O papel do backend
2. Cliente-Servidor
3. APIs & HTTP
4. Bancos de dados
5. Regras & Segurança
6. Ambiente & Ferramentas

1 O Papel do Backend

O que é

- Camada que o usuário NÃO vê
- Processa os dados que chegam do frontend
- Aplica a lógica de negócio
- Conversa com o banco de dados
- Conecta a interface do usuário aos dados

Frontend × Backend

Frontend = o que se vê

(tela, botões, cores)

Backend = o que processa

(dados, regras, segurança)

- Por que separar Backend e Frontend?
- Frontend valida também

2 Arquitetura Cliente-Servidor

CLIENTES



navegador · app · celular

Requisição →

← Resposta

SERVIDOR



processa e responde



Cliente

quem faz o pedido

- É de onde parte a requisição
- Ex.: navegador, app de celular
- Mostra o resultado ao usuário



Servidor

quem processa

- Computador remoto, sempre disponível
- Processa o pedido e acessa o banco
- Devolve a resposta ao cliente

- um único servidor atende muitos clientes simultaneamente
- Servidor “sempre” disponível

2 Ciclo Requisição / Resposta



- O cliente envia a requisição ao servidor
- O servidor processa e consulta o banco de dados
- O banco devolve os dados ao servidor
- O servidor monta e envia a resposta ao cliente
- HTTP é Stateless (cada requisição é independente)
- Um servidor pode ser cliente de outro servidor

3 APIs (Application Programming Interface)

O que é uma API

a porta de comunicação

- Regras para programas conversarem
- Define o que pedir e o que volta
- É a porta de entrada do backend
- Padroniza tudo via HTTP + JSON

API ≠ Backend

Backend = TODO o lado servidor:
lógica + banco + processamento

API = só a CAMADA que expõe
esse backend para fora

➜ **A API faz PARTE do backend**

3 Métodos HTTP: Express.js

GET · buscar

```
app.get('/users', (req, res) => {
  res.json(usuarios);
});
```

POST · criar

```
app.post('/users', (req, res) => {
  const novo = req.body;
  res.status(201).json(novo);
});
```

PUT · atualizar

```
app.put('/users/:id', (req, res)=>{
  // atualiza o usuário
  res.json(atualizado);
});
```

DELETE · remover

```
app.delete('/users/:id',(req,res)=>{
  res.status(204).end();
});
```

- A mesma url faz coisas diferentes conforme o método

3 Códigos de Status HTTP

200

`família 2xx`

Sucesso

Deu tudo certo, requisição concluída

404

`família 4xx`

Erro do cliente

Página/recurso não encontrado

500

`família 5xx`

Erro do servidor

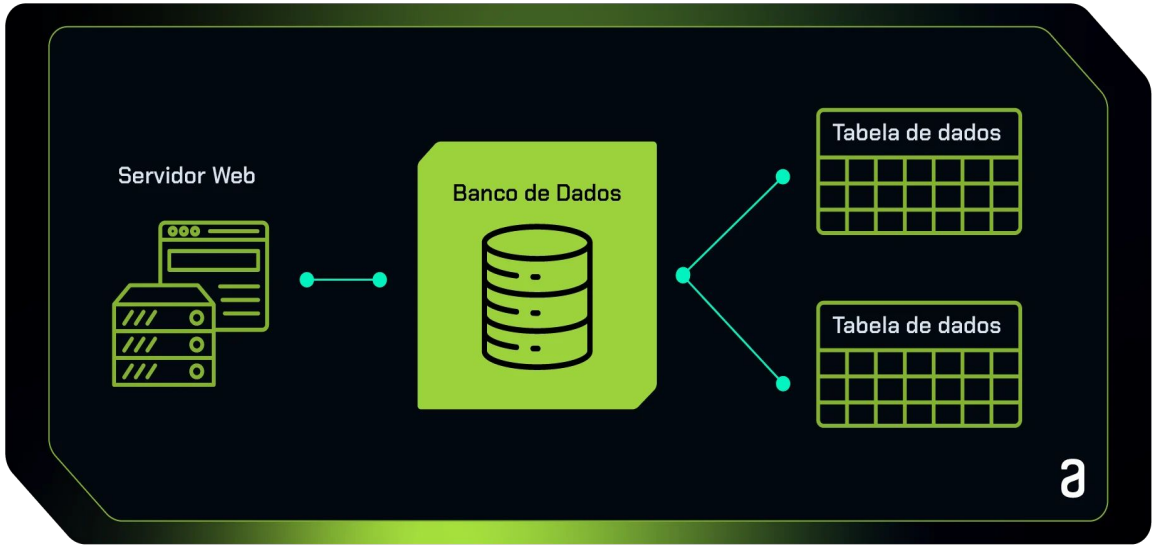
Algo falhou no lado do servidor

4 Bancos de Dados

Objetivo

guardar dados com segurança

- Armazenam a informação de forma permanente
- Mantêm os dados seguros e organizados
- Permitem buscar, gravar e atualizar dados
- Dois grandes tipos: SQL e NoSQL



- O cliente nunca fala diretamente com o banco

4 SQL × NoSQL

Relacionais (SQL)

tabelas com linhas e colunas

- Estrutura rígida e bem definida
- Dados organizados em tabelas
- Ótimo para dados consistentes
- Ex.: PostgreSQL, MySQL

Não Relacionais (NoSQL)

documentos e coleções

- Estrutura flexível, sem formato fixo
- Dados como documentos (ex.: JSON)
- Escala bem com grande volume
- Ex.: MongoDB, Redis

- Não tem melhor ou pior, são ferramentas para problemas diferentes

5 Regras de Negócio

Lógica de Negócio

o que pode e o que não pode

- As regras do mundo real dentro do código
- Decidem se uma ação é permitida ou não
- Fazem cálculos, validações e verificações
- Ficam no backend, longe do usuário

Exemplo: um Pix de R$ 100

1. Usuário pede para enviar R$ 100
2. Backend verifica se há saldo
3. Tem saldo? → faz a transferência
4. Não tem? → recusa e avisa

- tipo de coisa que não pode ser burlada pelo cliente

5 Autenticação × Autorização



Autenticação

QUEM é você?

- Confirma a identidade do usuário
- Login e senha, token, biometria
- Acontece PRIMEIRO
- Falhou? ⟶ erro 401



Autorização

O QUE você pode fazer?

- Define as permissões do usuário
- Usuário comum × administrador
- Acontece DEPOIS da autenticação
- Sem permissão? ⟶ erro 403

- cada chamada o backend valida o token (autenticação) e então checa as permissões (autorização)

6 Ambiente e Ferramentas

Node.js · Express

```
app.get('/', (req, res) => {
  res.send('Olá, mundo!');
});
```

Python · Flask

```
@app.route('/')
def home():
    return 'Olá, mundo!'
```

Java · Spring

```
@GetMapping("/")
String home() {
  return "Olá, mundo!";
}
```

C# · .NET

```
app.MapGet("/", () =>
  "Olá, mundo!");
```

6 Ambiente e Ferramentas



Sistema Operacional

- A base onde tudo é executado
- Coordena memória e arquivos
- Nos servidores, quase sempre Linux



Docker

- Empacota a app + o que precisa
- Roda igual em qualquer máquina
- Fim do "na minha funciona"



Nuvem

- Servidores alugados sob demanda
- Paga pelo uso e escala rápido
- Ex.: AWS, Google Cloud, Azure

Referências

GEEKHUNTER. O que é Back-end? Entenda o desenvolvimento do lado do servidor. Blog GeekHunter. Disponível em: https://blog.geekhunter.com.br/o-que-e-backend/

VIEIRA, N. Entendendo um pouco mais sobre o protocolo HTTP. Disponível em: https://nandovieira.com.br/entendendo-um-pouco-mais-sobre-o-protocolo-http

MOZILLA. HTTP - Métodos de requisição e Códigos de status. MDN Web Docs. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP

DIO. Dominando o Back-end: fundamentos do desenvolvimento de servidores e APIs. Digital Innovation One. Disponível em: https://www.dio.me/articles

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL Documentation. Disponível em: https://www.postgresql.org/docs/

MONGODB INC. MongoDB Manual. Disponível em:https://www.mongodb.com/docs/manual/

DOCKER INC. Docker Documentation. Disponível em: https://docs.docker.com/

OLIVEIRA, Danielle. Banco de Dados: o que é, principais tipos e um guia para iniciar. Alura, 8 fev. 2024. Disponível em: https://www.alura.com.br/artigos/banco-de-dados.